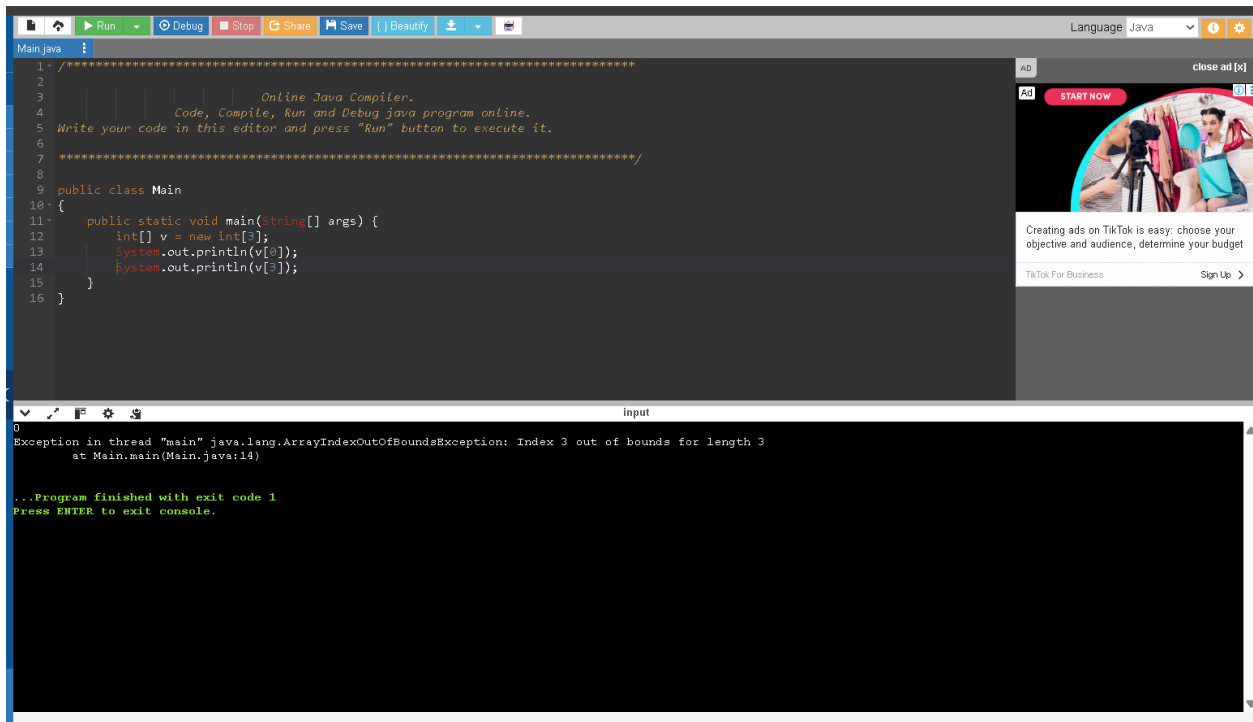


JAVA

a)



The screenshot shows an online Java compiler interface. The top section contains a code editor with the following Java code:

```
1  /*****  
2  
3      Online Java Compiler.  
4      Code, Compile, Run and Debug java program online.  
5      Write your code in this editor and press "Run" button to execute it.  
6  
7  *****/  
8  
9  public class Main  
10 {  
11     public static void main(String[] args) {  
12         int[] v = new int[3];  
13         System.out.println(v[0]);  
14         System.out.println(v[3]);  
15     }  
16 }
```

The bottom section shows the output console with the following error message:

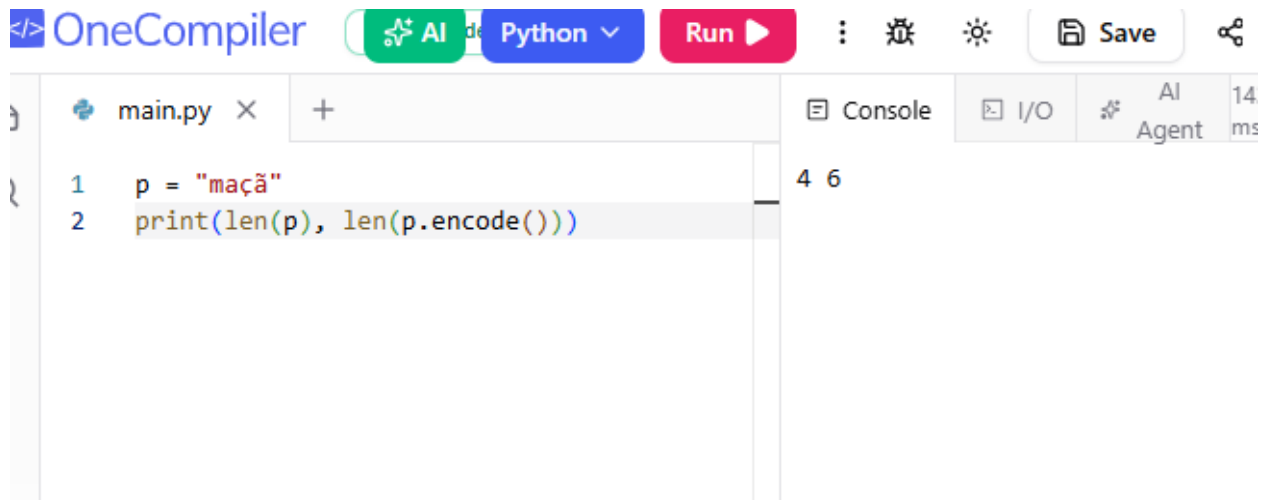
```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3  
at Main.main(Main.java:14)  
  
...Program finished with exit code 1  
Press ENTER to exit console.
```

The error message indicates that the program attempted to access an array element at index 3, which is out of bounds for an array of length 3. The console also shows that the program finished with exit code 1.

b) O erro ocorre na compilação. Pois alocamos um array de tamanho 3 e tentamos acessar o espaço 3, sendo que, no array temos os espaços: 0, 1, 2.

PYTHON

a)



The screenshot shows the OneCompiler Python IDE. The code editor contains the following Python code:

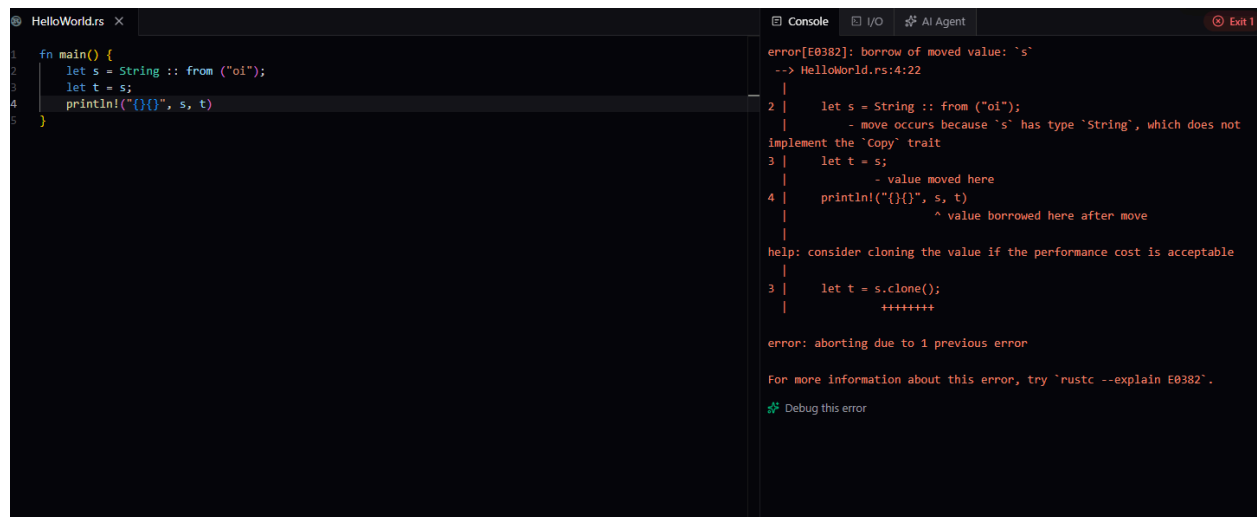
```
1 p = "maçã"
2 print(len(p), len(p.encode()))
```

The output console displays the result: 4 6. The first value, 4, is the length of the string "maçã". The second value, 6, is the number of bytes required to encode the string in UTF-8, as the 'ç' character is represented by two bytes.

b) Isso não é um erro, é um comportamento esperado pela linguagem. No primeiro `len()`, ele resulta no tamanho da string “maçã”, ou seja, 4. No segundo `len()`, ele faz com base em `p.encode()`, ou seja, resulta na quantidade de bytes ocupados pela string. Por terem caracteres especiais, o número difere um pouco do valor do primeiro `len()`

RUST

a)



The screenshot shows a Rust IDE with a compilation error. The code in the editor is:

```
fn main() {
    let s = String::from("oi");
    let t = s;
    println!("{}", s, t);
}
```

The error message in the console is:

```
error[E0382]: borrow of moved value: `s`
--> HelloWorld.rs:4:22
   |
2  |     let s = String::from("oi");
   |         - move occurs because `s` has type `String`, which does not
   |         implement the `Copy` trait
3  |     let t = s;
   |         - value moved here
4  |     println!("{}", s, t);
   |                  ^ value borrowed here after move
   |
help: consider cloning the value if the performance cost is acceptable
   |
3  |     let t = s.clone();
   |         ++++++

error: aborting due to 1 previous error

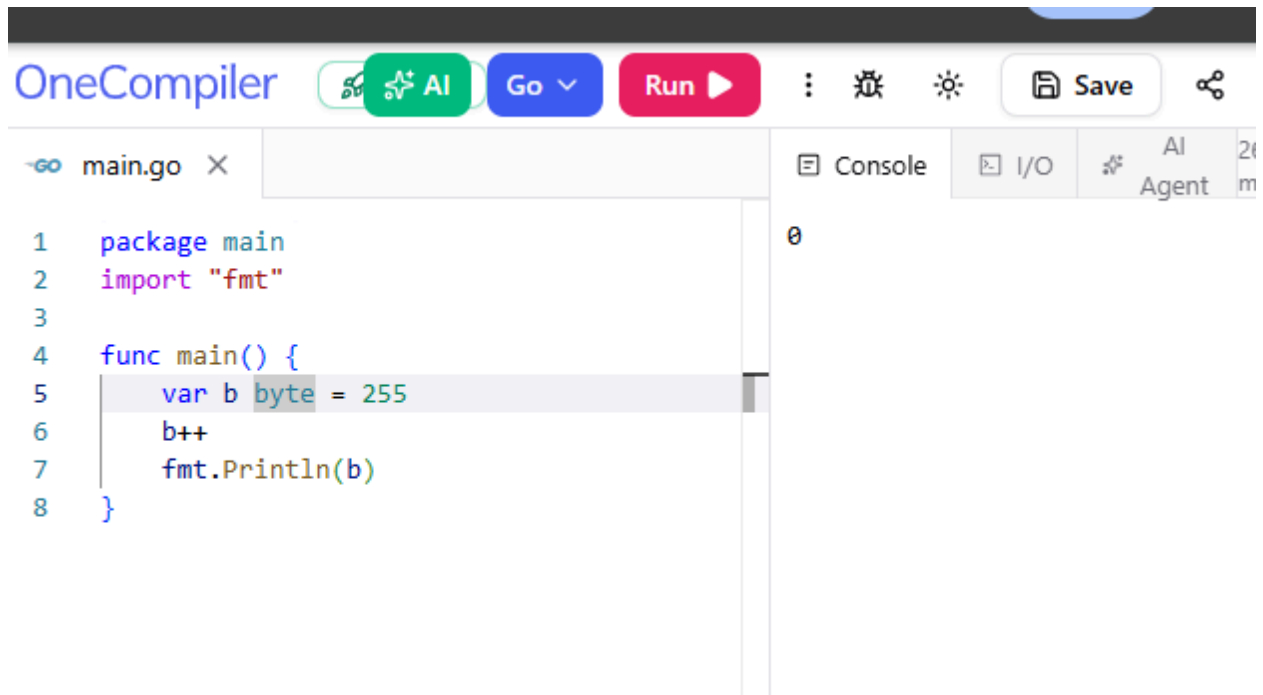
For more information about this error, try `rustc --explain E0382`.
Debug this error
```

The error occurs because `String` does not implement the `Copy` trait. When `s` is assigned to `t`, the value of `s` is moved. Subsequently, attempting to use `s` in the `println!` statement results in a borrow of a moved value.

b) A compilação não ocorre porque, na linha 3, “let t = s;”, o Rust move o valor de S para a variável t, e a variável s fica vazia. Portanto, quando tentamos acessar a variável s na linha 4, o Go exibe o erro “borrow of moved value: ‘s’”, pois a variável S esta vazia para evitar acesso duplo de memória.

GO

a)



```
OneCompiler  AI  Go  Run  Save  Share

main.go X

1 package main
2 import "fmt"
3
4 func main() {
5     var b byte = 255
6     b++
7     fmt.Println(b)
8 }
```

Console I/O AI Agent

0

b) O erro não é detectado pela linguagem. O motivo do problema é que o espaço máximo pra byte é 255, então ao atribuir 256 estouramos o espaço possível. Isso resulta em “0” no terminal.

JAVASCRIPT

a)



The screenshot shows a code editor with a file named 'index.js'. The code contains two lines: `console.log(0.1 * 3 === 0.3);` and `console.log(9007199254740993);`. The console output shows 'false' for the first line and '9007199254740992' for the second line.

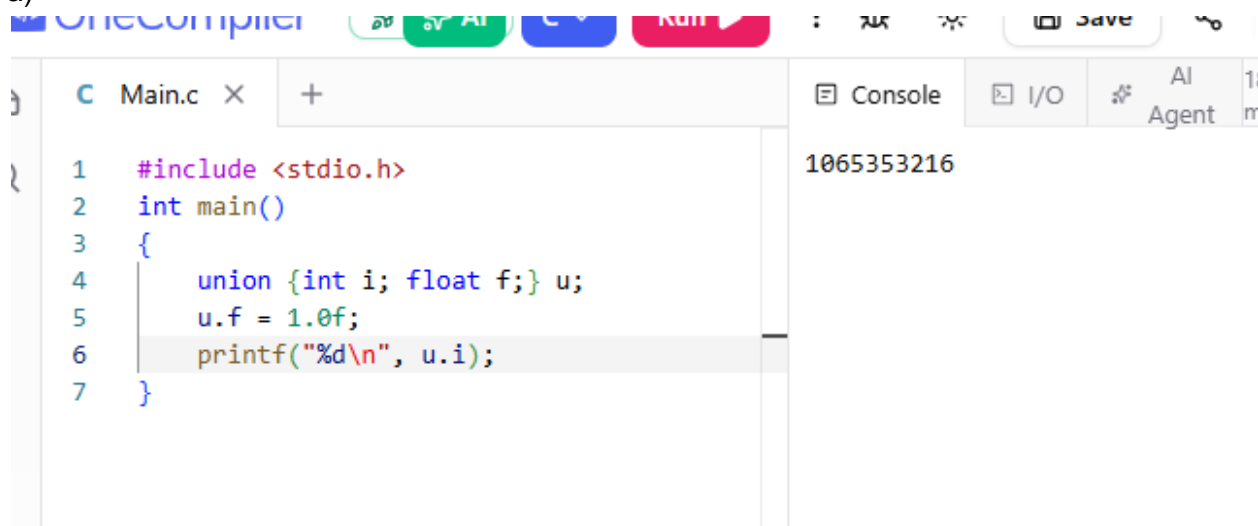
```
1 console.log(0.1 * 3 === 0.3);
2 console.log(9007199254740993);
```

Console output: false, 9007199254740992

b) O erro não é detectado pela linguagem. O motivo do problema no primeiro é a falta de precisão para lidar com cálculo de ponto flutuante em JS. No segundo, o inteiro estoura o tamanho que pode ter, por isso não é representado da maneira como está escrito.

C

a)



The screenshot shows a code editor with a file named 'Main.c'. The code defines a union with an integer 'i' and a float 'f'. It sets 'u.f' to 1.0f and then prints 'u.i'. The console output shows the integer value 1065353216.

```
1 #include <stdio.h>
2 int main()
3 {
4     union {int i; float f;} u;
5     u.f = 1.0f;
6     printf("%d\n", u.i);
7 }
```

Console output: 1065353216

b) O erro não é detectado pela linguagem. Quando usamos union, podemos atribuir diferentes tipos de valores. No caso, atribuímos um float, mas depois tentamos imprimir um inteiro, o que resulta em lixo